

Hyperparameters

Parameters: Some metrics that is learnt during training process are called Parameters.

Such as weight, bias

Hyperparameters: are the training / learning algorithm related parameters.

Hyperparameters manually specified.

When designing an NN, there are a number of hyperparameters to select like NN model architecture, the number of hidden layers, the number of neurons per each hidden layer, the learning rate, Number of iterations, Number of epochs, Choice of activation function, Batch size (All data at once / single data point at once / Minibatch)

Hyperparameters (Contd..)

Hyperparameters that control the parameters.

User need to tell the training / learning algorithm.

Hyperparameters are the parameters whose values are decided before training of the model begins.

There are no efficient algorithm to select optimal or best values of hyperparameters.

So the optimal values for the hyperparameters are determined using trial and error method by adjusting their values.

This process of determining the values of hyperparameters is called 'hyperparameter tuning.'

There are many hyperparameters to optimize when working with ANNs.

The objective here is to know the procedure we should follow to determine the best hyperparameters.

Hyperparameters affect the quality and speed of the training.

Normalization

One of the important issues in NN is that the training of the network takes a long time for deep networks.

By backpropagation, the NN learn how much error it did and correct themselves, i.e, correct their “weights” and “biases”.

By this, NN learn the problem to produce correct outputs for the given inputs.

Backpropagation involves computing gradients for each layer and propagating it backward, hence the name.

But during backpropagation of errors to the weights and biases, we'll face a undesired property of Internal Covariate Shift. This makes the network too long to train.

Normalization (Regular normalization)

In a dataset, all the features may not be in same range. It takes lot of time to train for these kind of datasets.

We normalize the input layer by adjusting and scaling the features.

Eg: When we have features from 0 to 1 and some from 1 to 1000, we should normalize them to speed up learning.

Same thing also for the values in the hidden layers, that are changing all the time, and get 10 times or more improvement in the training speed.

Batch normalization reduces the amount by what the hidden unit values shift around.

Normalization is a rescaling of the data from the original range so that all values are within the range of 0 and 1.

Batch Normalization

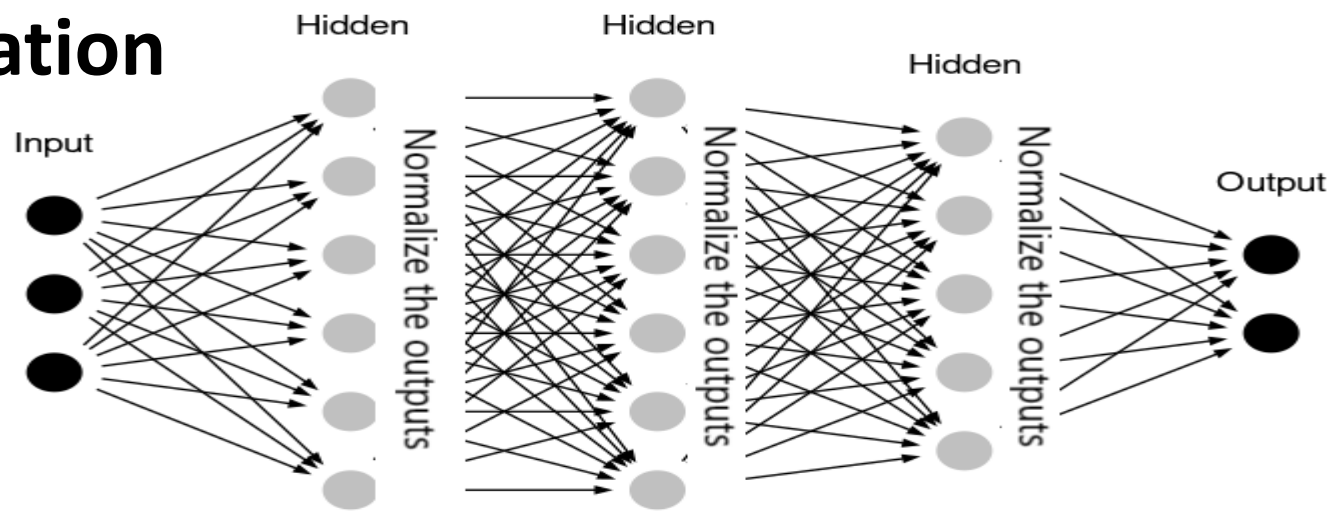


Fig: Batch Normalization

To reduce this problem of internal covariate shift, Batch Normalization adds Normalization “layer” between each layers. In Batch normalization we normalize all the outputs of all the layers. See the dia.

Normalization brings all the inputs centered around 0.

This way, layers in the network can learn from the back-propagation simultaneously, without waiting for the previous layer to learn.

This fastens up the training of networks.

Epochs take longer time due to the amount of computations but convergence will be faster. This is because of data presented is now normalized.

Achieves same accuracy as that of non-normalization with less epochs. This leads to the better performance.

No need of preprocessing step of standardization of data at the input as seen with normalization.

Batch normalization reduces the need for regularization.

Batch Normalization normalizes the values with a four-step process:

- i) Computing the mean of your minibatch.
- ii) Computing the variance of your minibatch.
- iii) Normalizing the value.
- iv) Scaling and shifting.

i) & ii) Computing mean μ and variance :

The first two steps are simple mathematical procedure and are very common as well as required in a normalization step: computing the mean and variance

iii) Normalizing:

The next step, in which the expected distribution is (0,1) as long as samples have the same distribution

iv) Scaling and shifting: Finally, scaling and shifting the normalized input value.

Batch Normalization may help to reduce the issues with your neural network. By reducing the distribution of the input data to (0, 1), batch Normalization is theoretically expected to reduce what is known as the “internal covariate shift”, resulting in faster learning.

Regularization:

Underfitting means model is not learning well. The model is not fit to learn the presented data.

Overfitting means model is learning the data presented to it and memorizes. But model is not flexible. Model fails when unseen data is presented.

So a good fit model is required for a presented dataset.

So to find a Good fit model we need regularization. Regularization avoids overfitting. There are different ways of regularization:

L1 regularization:

$$\text{Cost} = \text{Loss}(y, \hat{y}) + \lambda \sum |W|$$

Note: $\sum |W|$ is added as penalty. λ that controls the penalty strength.

L2 regularization:

$$\text{Cost} = \text{Loss}(y, \hat{y}) + \lambda \sum W^2$$

$$\text{Loss}(y, \hat{y}) = \text{MSE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

How does batch normalization work?

To increase the stability of a neural network, batch normalization normalizes the output of a previous activation layer by subtracting the batch mean and dividing by the batch standard deviation.

After this scale of activation outputs, the weights in the next layer may be optimal. SGD undoes this normalization if it's a way for it to maximize the loss function.

If so, batch normalization adds two trainable parameters to each layer, so the normalized output is multiplied by a “standard deviation” and add a “mean”.

In other words, batch normalization lets SGD do the denormalization by changing only these two weights for each activation, instead of losing the stability of the network by changing all the weights.

How does batch normalization work?

To increase the stability of a neural network, batch normalization normalizes the output of a previous activation layer by subtracting the batch mean and dividing by the batch standard deviation.

Regression

Regression refers to functions that attempt to predict a real value output.

This type of function estimates the dependent variable by knowing the independent variable.

Linear regression:

If all layers use an “identity” activation function, then a multilayer network can be shown to simplify to linear regression.

“A neural network with a linear activation function is simply a linear regression model”.

Multi-layer network that uses only the identity activation function in all its layers reduces to a single-layer network performing linear regression.

What is logistic or nonlinear regression?

Nonlinear regression is a form of **regression** analysis in which data is fit to a model and then expressed as a mathematical function. ... **Nonlinear regression uses** exponential functions, logarithmic functions, trigonometric functions, and other fitting methods.

Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN)

form of images or video frames

face detection →

face recognition.

256×256

512×512

1024×1024



Map human brain to CNN

Divided in 4 parts:

Cerebral Cortex

Visual Cortex → for seeing the image

multiple layers V_1, V_2, V_3, V_4, V_5 & V_6 to process the image info

is involved in extracting some features from image observed

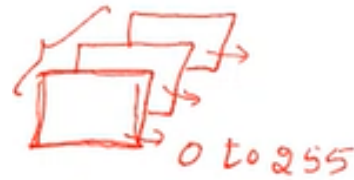
Convolutional operation in CNN

Convolutional operation:

Grey scale image $\rightarrow$ 0 to 255 6×6

Colored image $\rightarrow$ R G B $6 \times 6 \times 3$

Normalizing the image



0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

6×6

1	0	-1
2	0	-2
1	0	-1

3×3
filter

0	-4	-4	0
0	-4	-4	0
0	-4	-4	0
0	-4	-4	0

4×4

$\Rightarrow$

0	1	1	0
0	1	1	0
0	1	1	0
0	1	1	0

min = -1 max = 0
min-max scalar

$$0 \times 1 + 0 \times 0 + 0 \times -1 + 0 \times 2 + 0 \times 0 + 0 \times -2 + 0 \times 1 + 0 \times 0 + 0 \times -1 = 0$$

$$0 \times 1 + 0 \times 0 + 1 \times -1 + 0 \times 2 + 0 \times 0 + 1 \times -2 + 0 \times 1 + 0 \times 0 + 1 \times -1$$

$$= -1$$

$$-2$$

$$-1$$

$$= -4$$

Padding in CNN:

0	0	0	1	1	1
0	0	▽	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

6x6 $\rightarrow$ 8x8

Zero padding : insert 0 in all extra cells

2) nearest neighbor value padding

$$n \times n = 6 \times 6$$

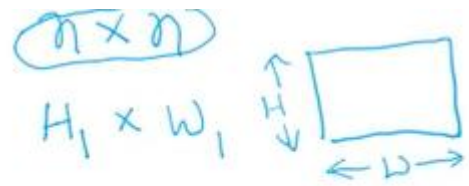
$$P = 1$$

6×6

$$n + 2p - f + 1$$

$$6 + 2 \times 1 - 3 + 1 = 6 \rightarrow 6 \times 6$$

6x6 $\rightarrow$ 8x8



$H_2 \times W_2$

$$H_2 = \frac{(H_1 + 2P - f)}{S} + 1$$

$$W_2 = \frac{(W_1 + 2P - f)}{S} + 1$$

No. of filters:

filter size f

Stride S

Padding P

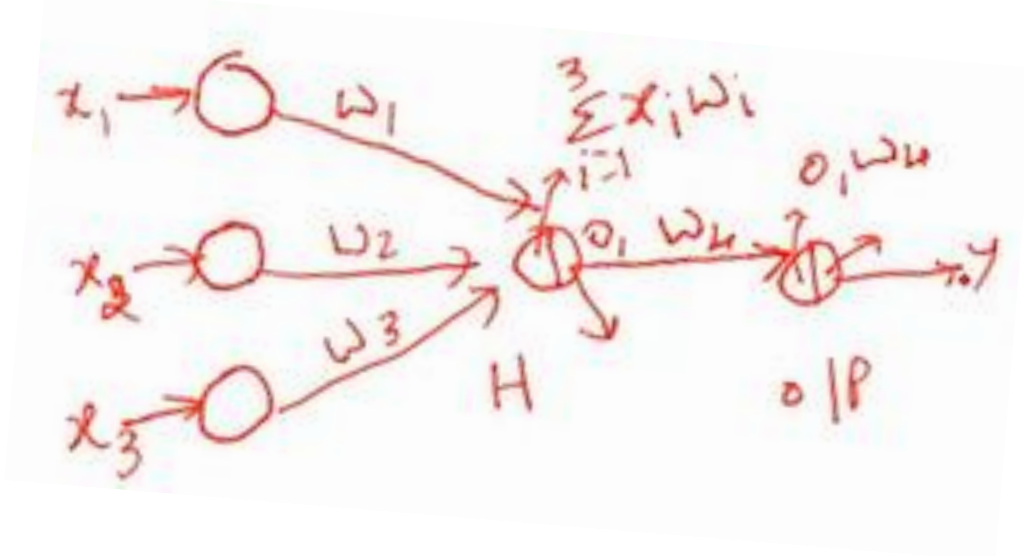


Epoch

Layers in the NN

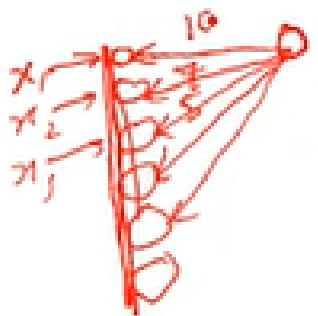
Neurons on each layer

Additional information on operations of CNN

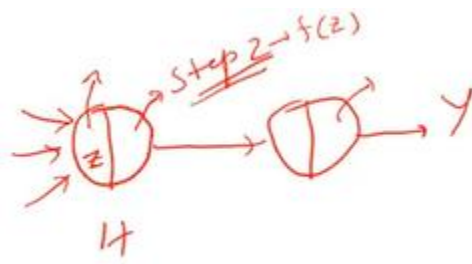


Learn the filter(s)

values of the filters are updated during BP



ReLU layer



$$\begin{aligned} f(z) &= 0 && \text{for } z < 0 \\ &= z && \text{for } z \geq 0 \\ & && (\text{or}) \\ &= \max(z, 0) \end{aligned}$$

-1	0	5	4
3	-4	-8	3
1	4	6	-5
-2	-5	4	1



0	0	5	4
3	0	0	3
1	4	6	0
0	0	4	1

Pooling Layer in CNN

0	0	5	4
3	0	0	3
1	4	6	0
0	0	4	1

0	0	5	4
3	0	0	3
1	4	6	0
0	0	4	1

o/p

3	

0	0	5	4
3	0	0	3
1	4	6	0
0	0	4	1

3	5

0	0	5	4
3	0	0	3
1	4	6	0
0	0	4	1

3	5
4	6

$$(n-f)/2 + 1$$

$$(4-2)/2 + 1 = 1 + 1 = 2$$

$$2 \times 2$$

Flattening layer

1	1	0
4	2	1
0	2	1

1

1	1	0
4	2	1
0	2	1

Flattening layer

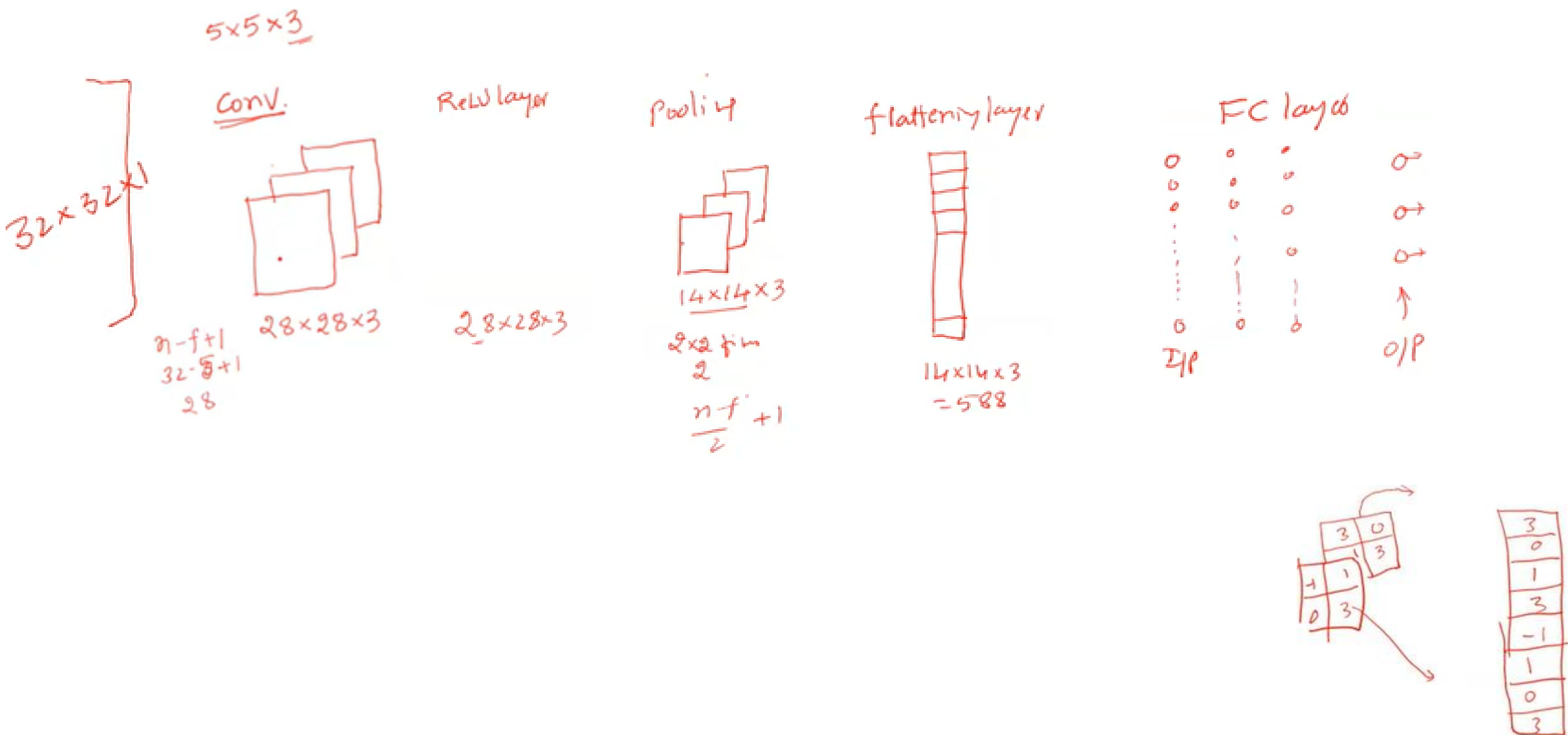
1
1
0
4
2
1
0
2
1

→0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
I/p	H1	H2	H3	O/p

FC layer

$\rightarrow 0$	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
<u>I/p</u>	H1	H2	H3	O/p

An example case of CNN



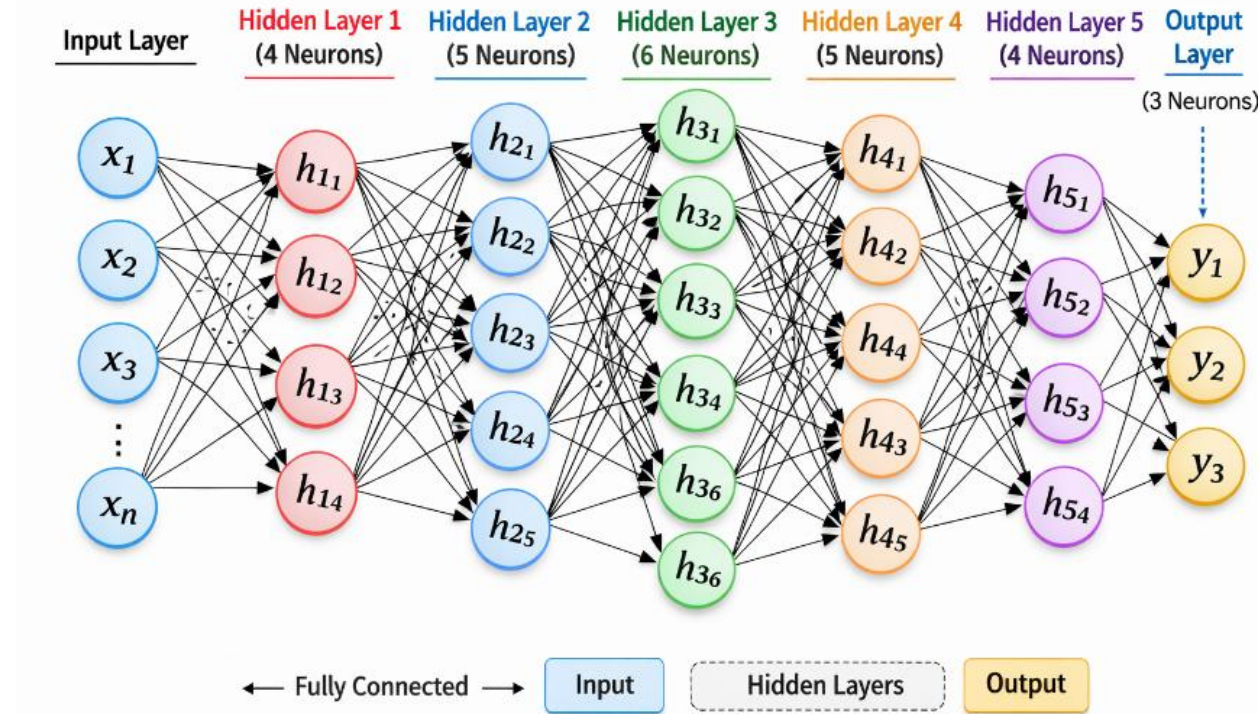
Some example case analysis of CNN

Additional information on NNs

How many hidden layers are to be there in a NN? And how many neurons on each HL?

ResNet (Residual Network)

Difficulty with the DNN is hard to train the network because of vanishing gradient problem.

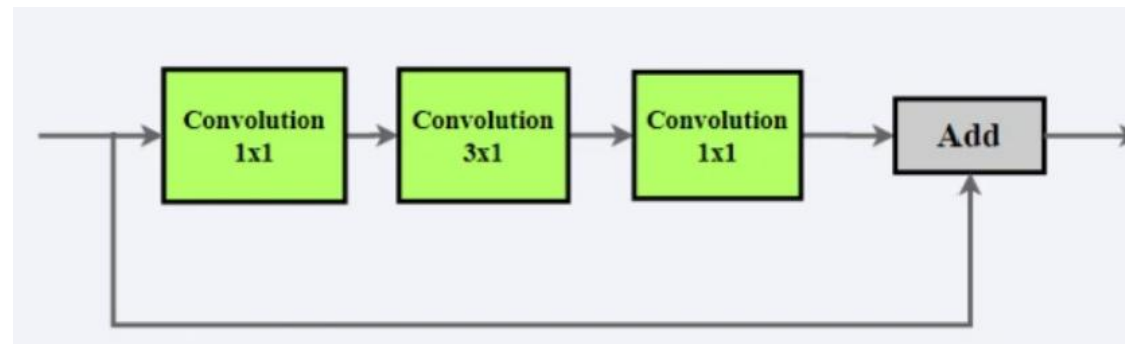


While updating the weights during BP, the vanishing gradient problem. Derivative of L wrt input is the product of the intermediate derivatives. So what happens is that the product leads to very very small value. ResNet are DNN but they do not meet the vanishing gradient problem.

ResNet use the concept of 'Skip Connection' which means that add the original input to the output of the Convolutional layer.



Residual block:

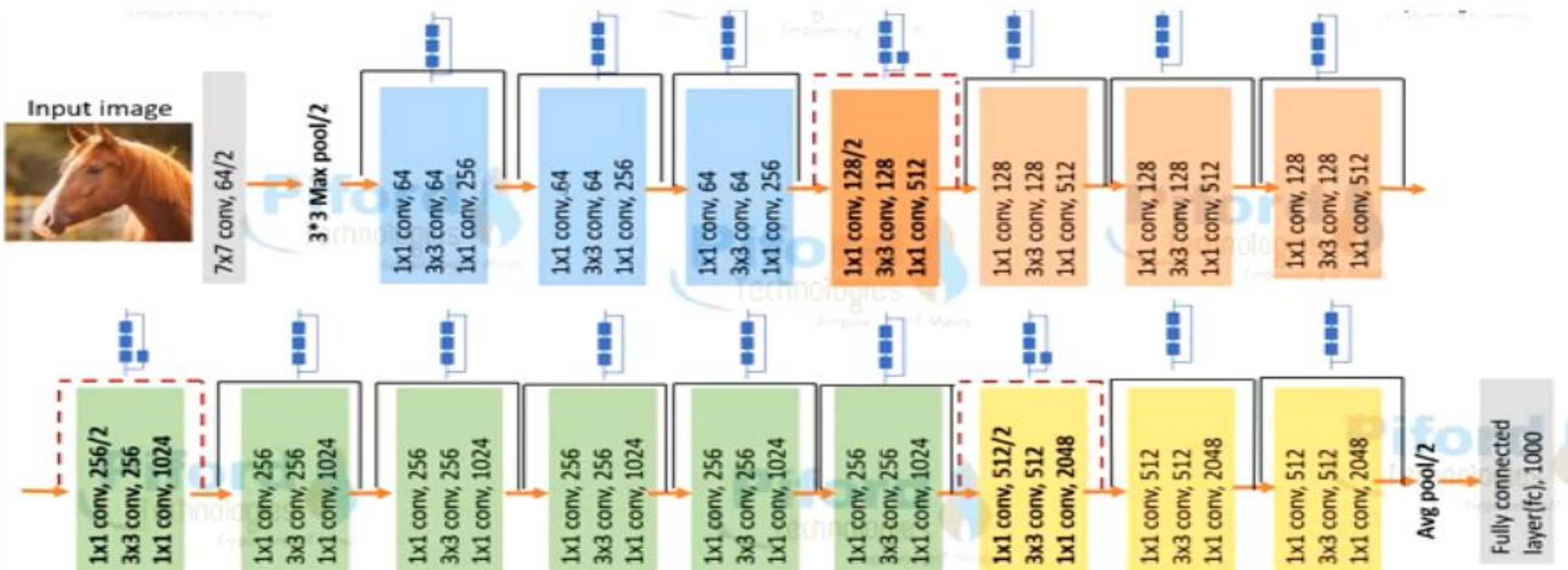
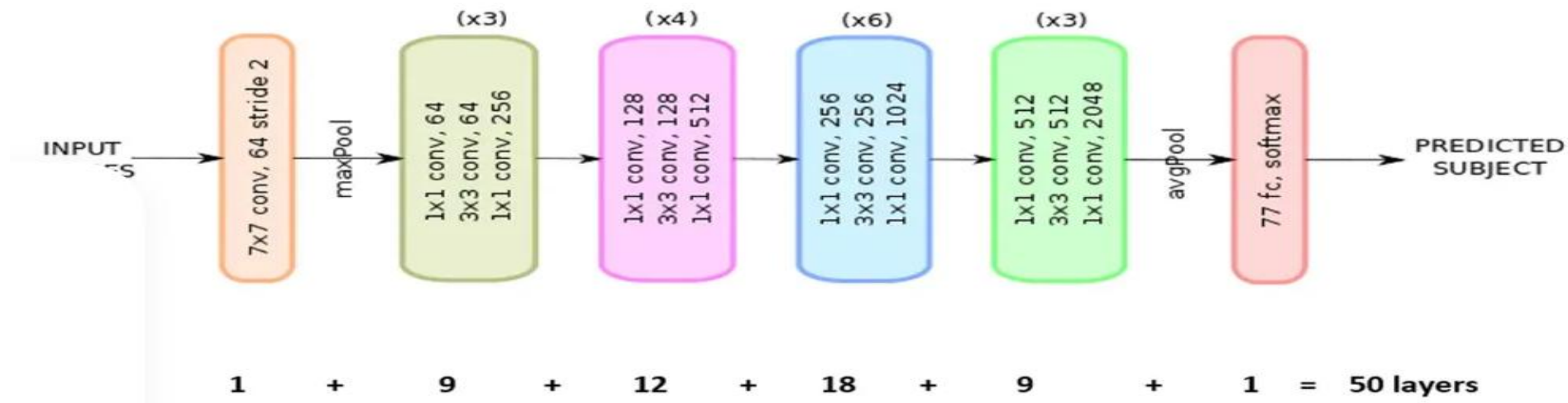


$$Y = f(X) + X$$

Where X is the actual input added to the output. Main logic here is we want to make $f(x) = 0$ so that $Y = X$. In other words we want to increase the overall accuracy.

In normal network, we learn from Y , but in ResNet we learn from $f(X)$ and our target is to make $f(X) = 0$ then only we can make output = input (I.e. $Y = X$).

ResNet 50:



Filter size is 7 X 7

So output image size is $(300 + 2 \times 3 - 7)/2 + 1 = 150 \times 150 \times 64$

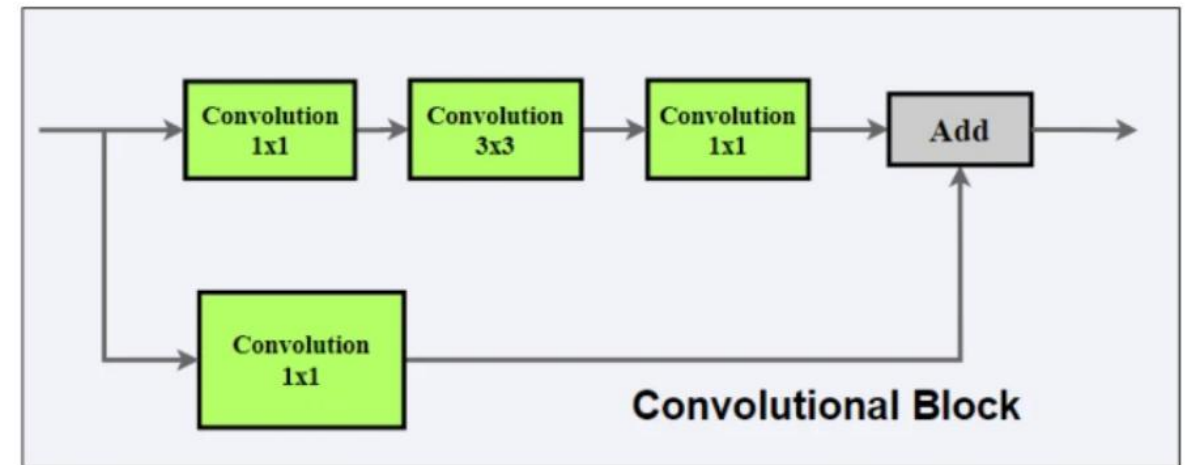
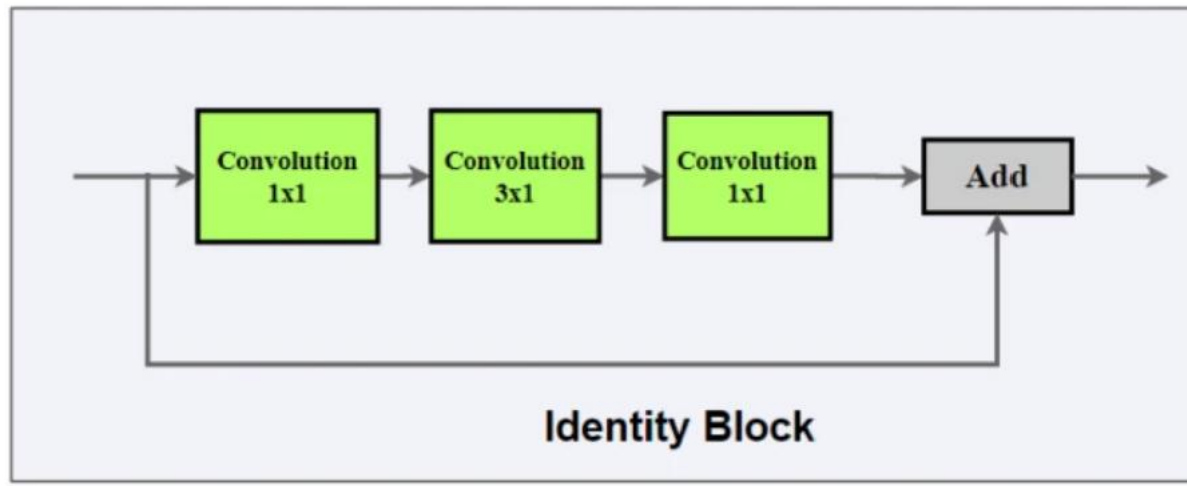
Max pool layer size 3 X 3 with stride =2 and padding 1.

So output from max pool layer image size $(300 + 2 \times 1 - 3)/2 + 1$ i.e. $75 \times 75 \times 64$

Skip connection:

Solid means identity block

Dotted means convolutional block.



Now how the convolutional layer in shortcut path works:

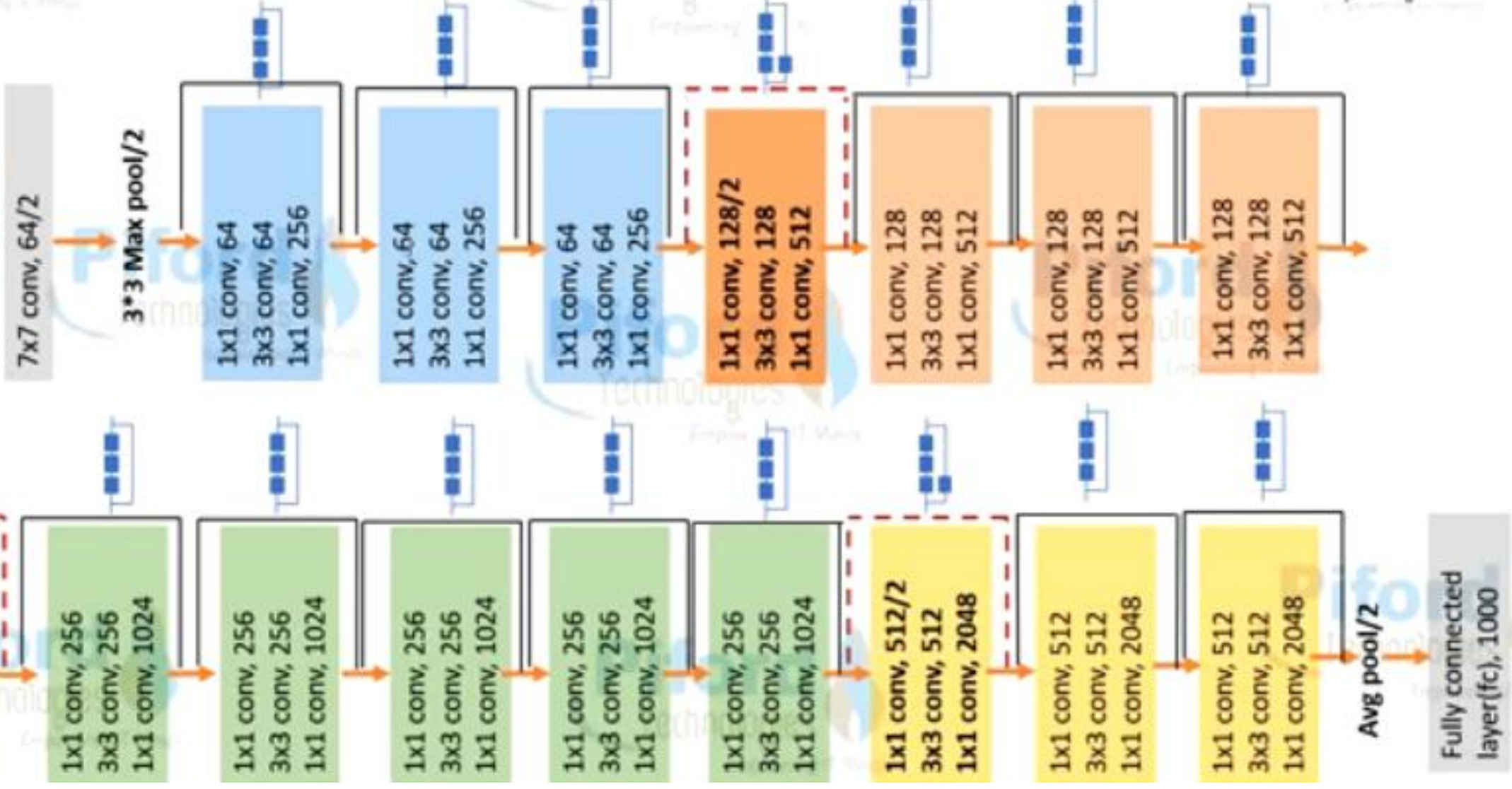
There are 2 options for matching the output size:

- a) Padding the input
- b) Perform 1 X 1 convolution

Now with method 2, convolutional layer in shortcut path will work with input image size being 56 X 56 X 64

$$(56 + 2 \times 0 - 1) / 2 + 1 = 28 \times 28$$

So now we can add the input and the output being the same size as 28 X 28.



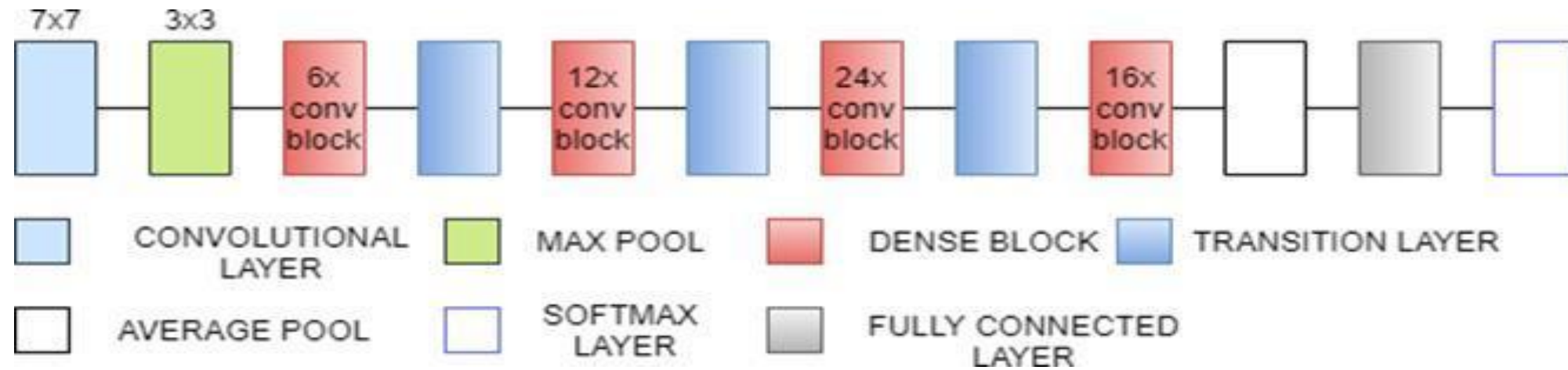
AlexNet

DenseNet

DenseNet is for image classification and is developed to improve the accuracy of the model by handling vanishing gradient problem.

Due to the large number of layers in DNN, the information vanishes before it reaching the destination during back propagation.





DenseNet has different variants. We address DenseNet-121 means this convolutional NN has 121 layers in it.

Input goes to CNN where we use filter of size 7 X 7 with stride 2 followed by pooling layer 3 X 3 with stride 2.

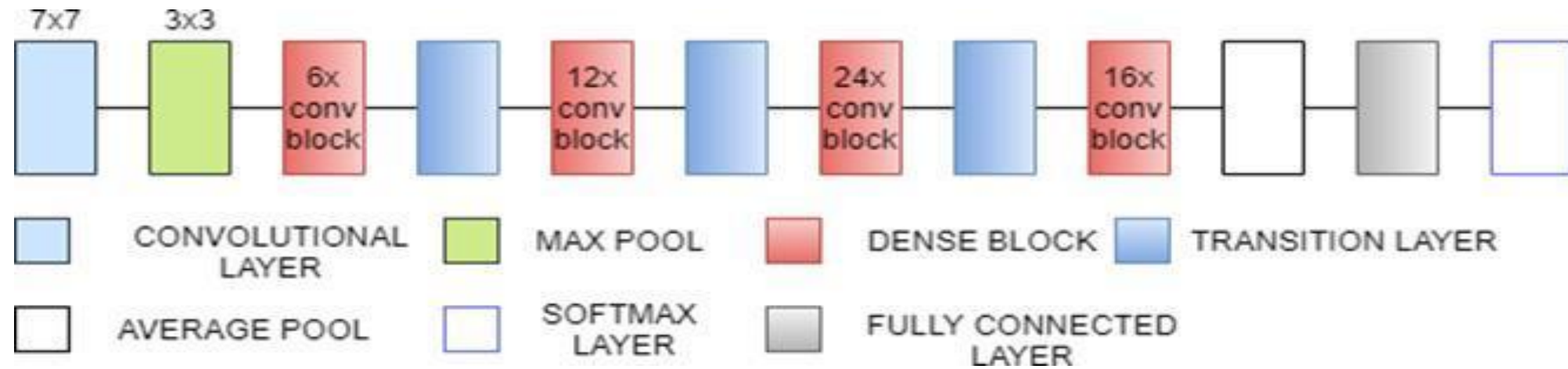
DenseNet have Dense blocks in it. Here with this, we have 4 Dense blocks. In each of the Dense blocks we have different number of Convolutional layers.

Dense block 1 has 6 convolution layers

Dense block 2 has 12 convolution layers

Dense block 3 has 24 convolution layers

Dense block 4 has 16 convolution layers



After every Dense block there is a transition layer

In each transition layer, has convolution layer of size 1×1 followed by average pooling layer of size 2×2 with stride 2.

Input

Conv 7x7 / stride 2

Pool 3x3 / Stride 2

Dense Block 1

Conv 1x1

Conv 3x3

x6

Transition Layer

Conv 1x1

Avg Pool 2x2 /
Stride 2

Dense Block 2

Conv 1x1

Conv 3x3

x12

Transition Layer

Conv 1x1

Avg Pool 2x2 /
Stride 2

Dense Block 3

Conv 1x1

Conv 3x3

x24

Transition Layer

Conv 1x1

Avg Pool 2x2 /
Stride 2

Dense Block 4

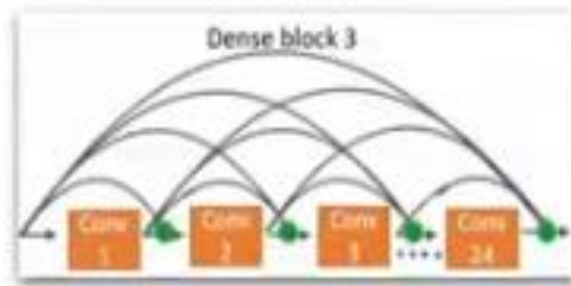
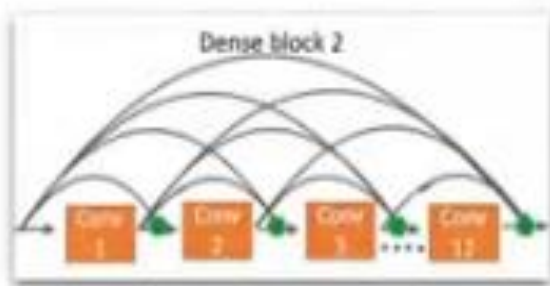
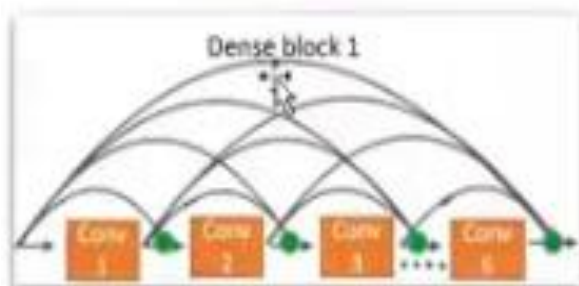
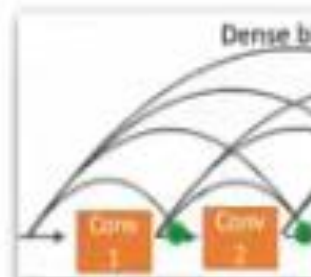
Conv 1x1

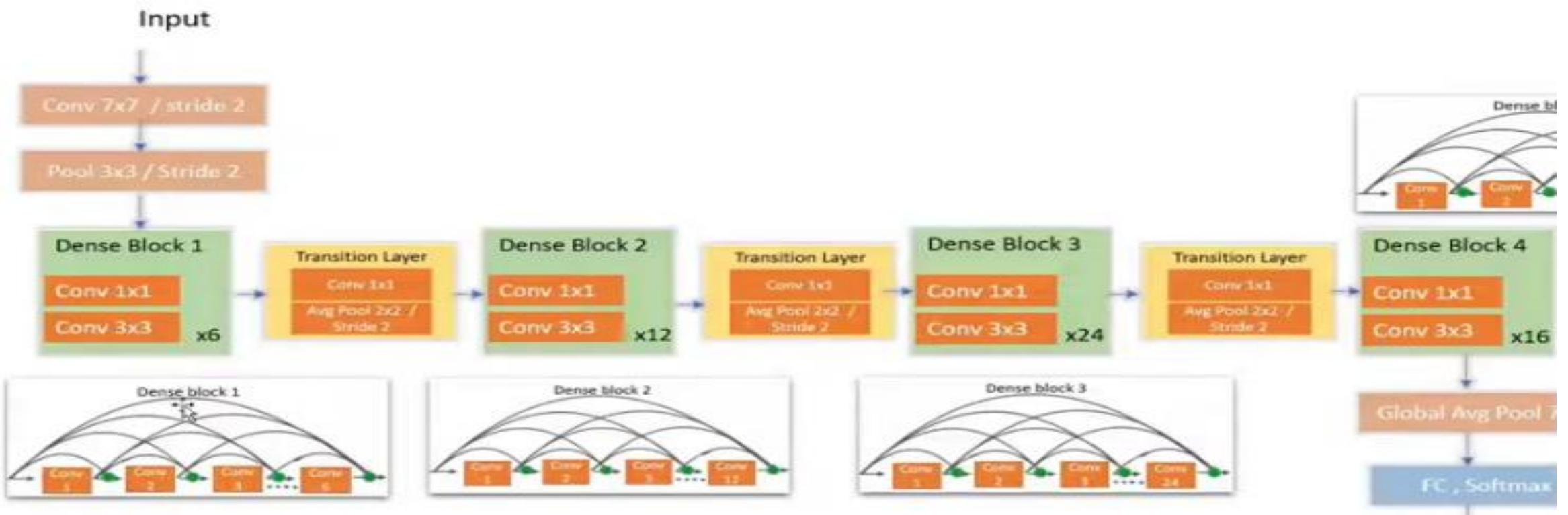
Conv 3x3

x16

Global Avg Pool 7

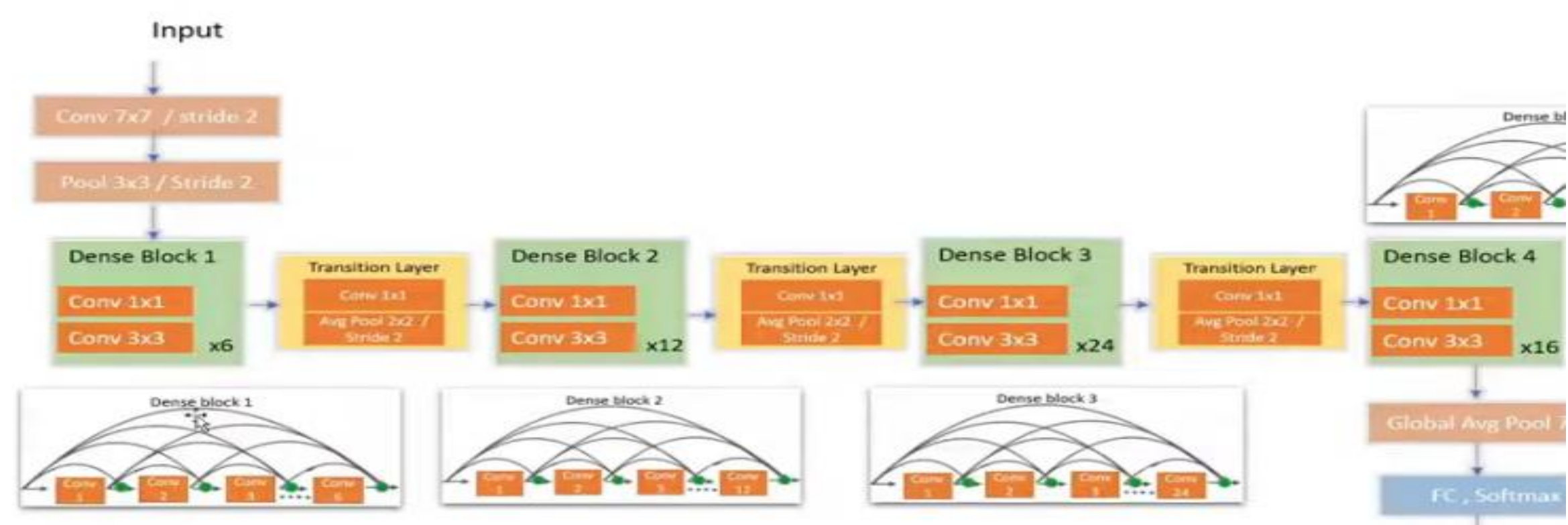
FC, Softmax





Each Dense block when unfolded then..

Each convolutional layer is connected to every other convolutional layer inside a dense block.

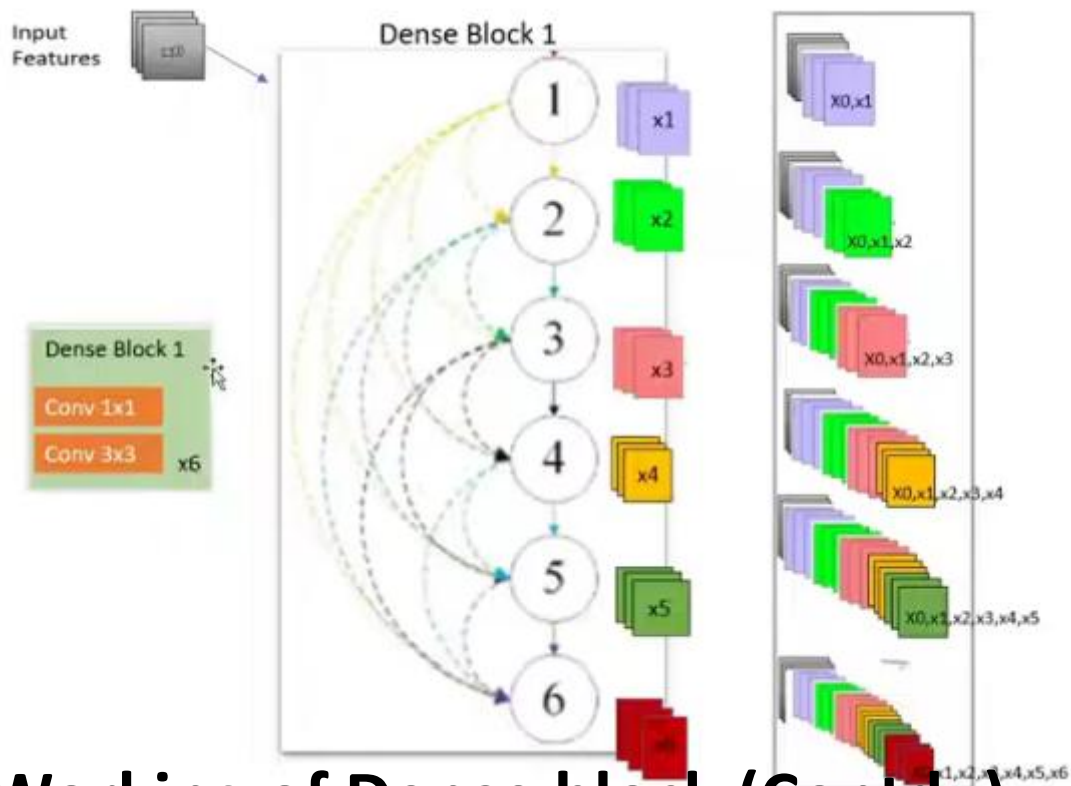


Working of Dense block:

In Dense block 1, there are 6 convolutional layers. Each layer will receive the feature map from the previous layer.

Conv layer 1 is going to provide its feature map as input to the other layers.

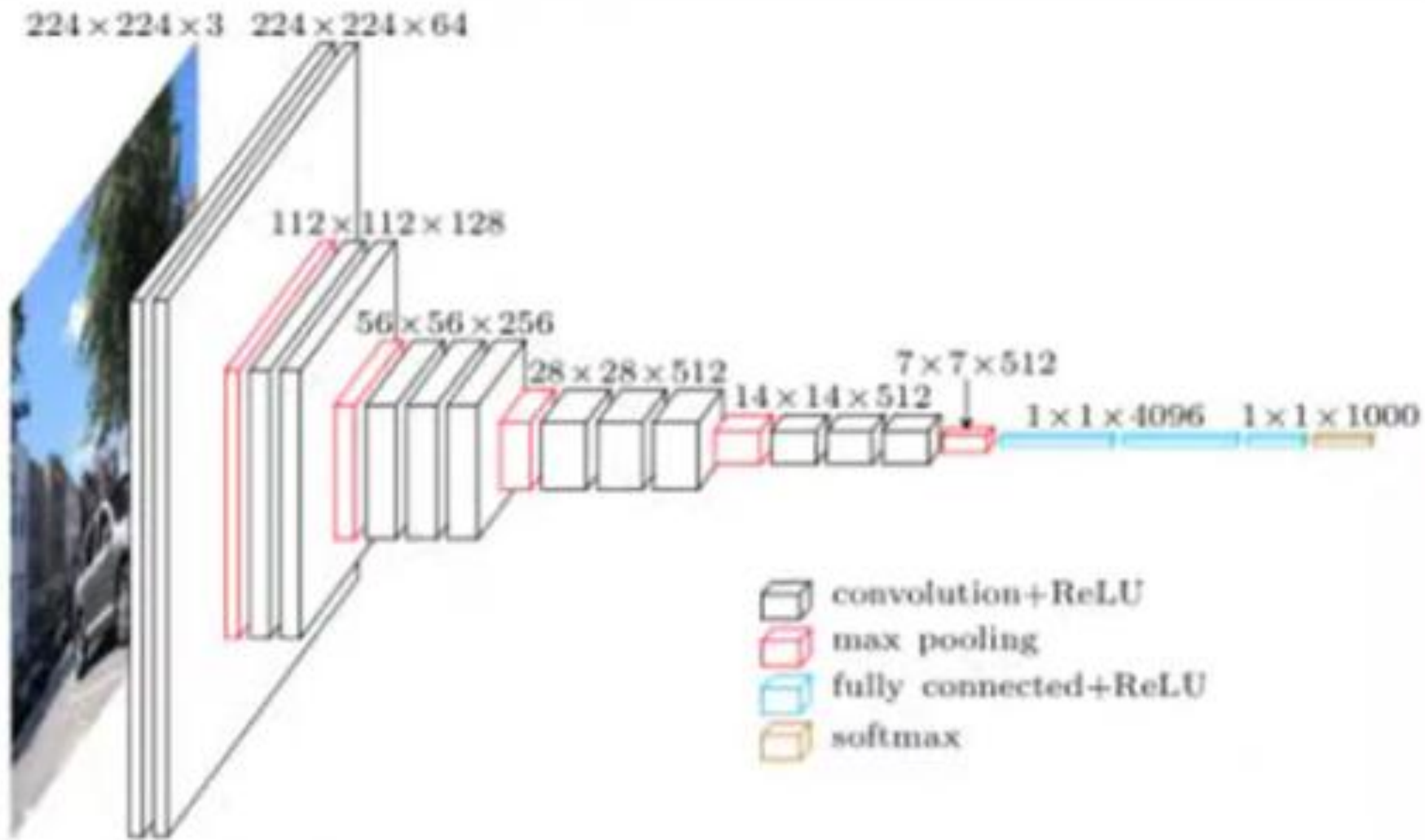
Conv layer 2 is going to provide its feature map as input to the other layers and so on. While it is extracting the feature map it is also going to concatenate it with the Conv layer 1 feature map.



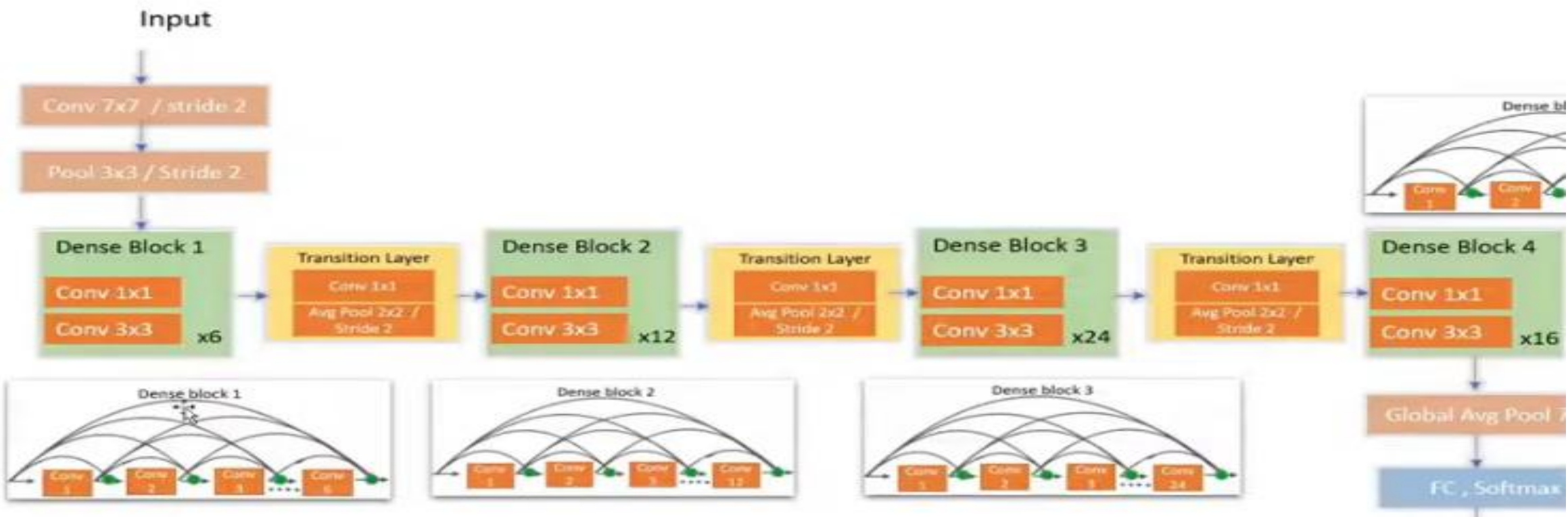
Working of Dense block (Contd..):

..

Note: Concatenation can be done only if all the feature map size is same.

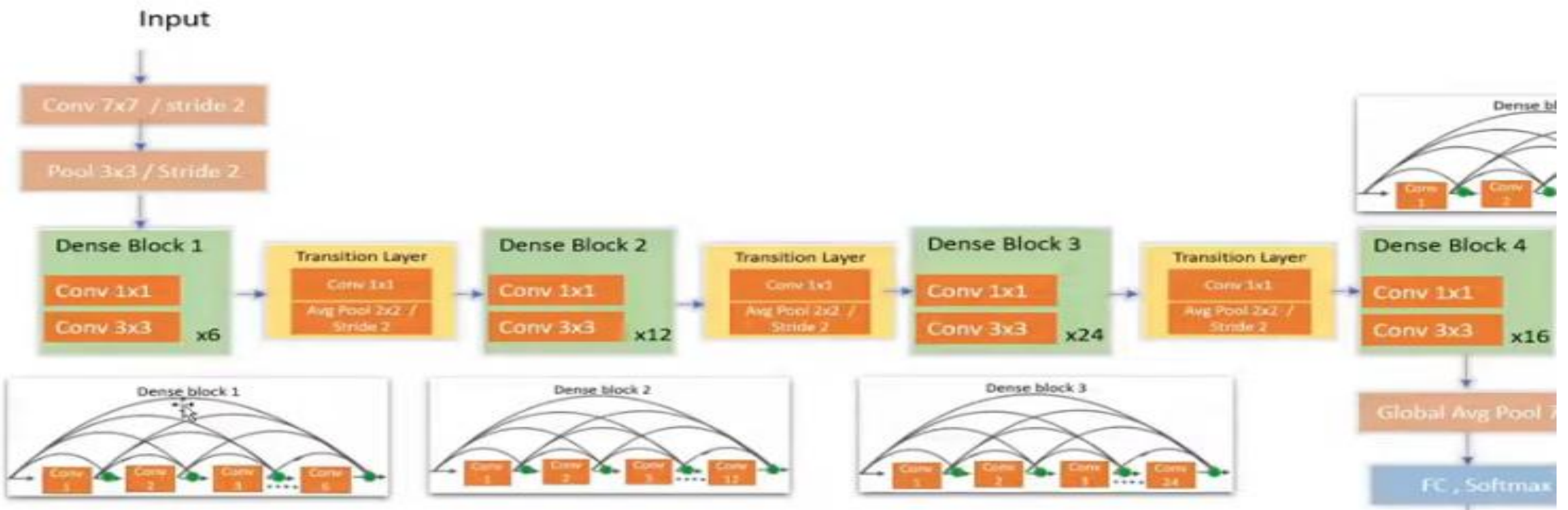


In above diagram with input size $224 \times 224 \times 3$ image is down sampled to the size of $7 \times 7 \times 512$. Every convolutional NN down samples the input image.



But in our Dense block, concatenation..
 If down sampling happens between the conv layers then..
 Down sampling is not done within the dense block.

..

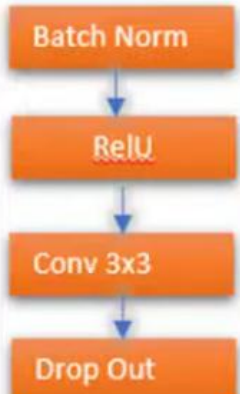


Down sampling is not done within the dense block, instead it is done through transition layer.

..

..

Each convolution layer is consist of :



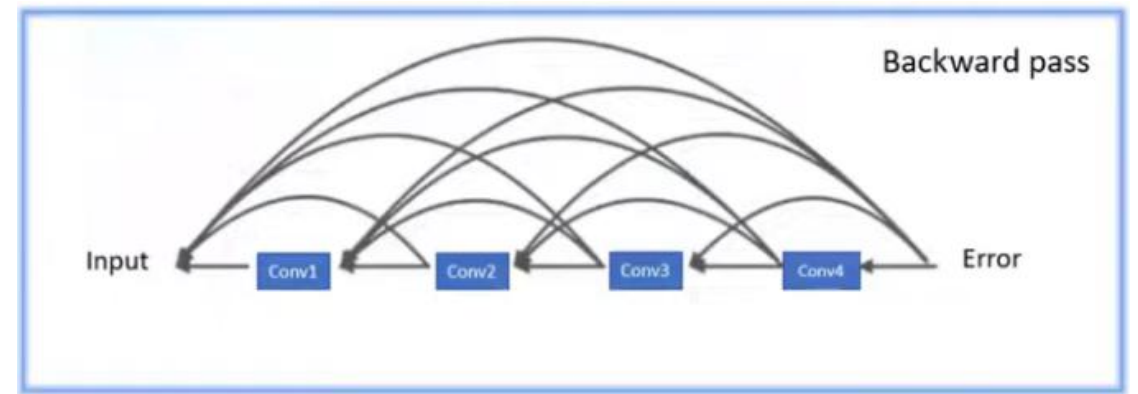
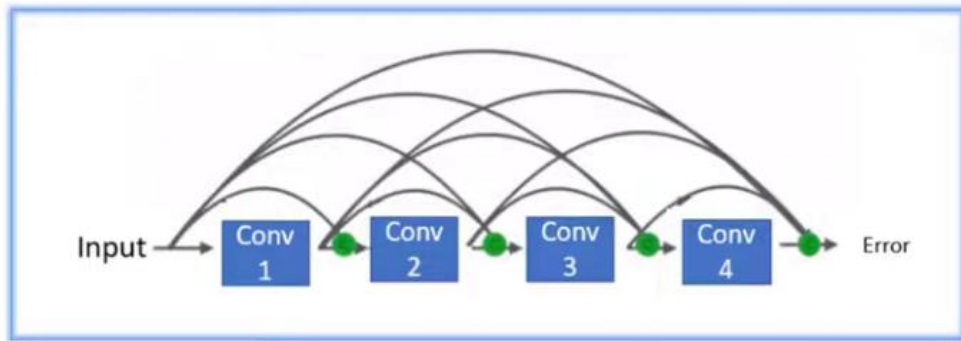
Each Conv layers in any Dense block will have Batch normalization + ReLU + conv Kernel 3 X 3 and may be dropout layer.

Transition Layer consist of:



Illrly, in transition layer, we do Batch normalization + ReLU + conv Kernel 1 X 1 and may be dropout layer + pooling layer 2 X 2.

During forward propagation every layer is connected to every other layer



And after calculating the error, now in back propagation, error / loss gradient sent to every layer. i.e. in DenseNet, each layer has direct access to the loss gradients which means that info flow will be good.

This way, the previous layers get benefit of accessing the loss gradients directly from the output and hence no vanishing gradient issues.

